

Table of Contents

Chapter 4: Control Structures in Python	2
Complete Question Bank.....	3
PART A — Multiple Choice Questions (MCQs)	3
Section 1: Decision Making (4.1)	3
Section 2: Looping Constructs (4.2)	6
Section 3: Libraries in Python (4.3)	10
Section 4: Lists in Python (4.4)	12
Section 5: Testing and Debugging (4.5).....	15
MCQ Answer Key	18
PART B — Short Questions.....	18
Section 1: Decision Making	18
Section 2: Looping Constructs.....	20
Section 3: Libraries	22
Section 4: Lists	23
Section 5: Testing and Debugging	25
PART C — Long Questions.....	27
Quick Reference Summary	42

Chapter 4: Control Structures in Python

Complete Question Bank

How to use this file:

- **MCQs** — Circle or highlight the correct option. Answers are listed at the end of the MCQ section.
 - **Short Questions** — Write 2–4 sentences per answer. Each answer should be concise and to the point.
 - **Long Questions** — Write structured, detailed answers with code examples where asked.
-

PART A — Multiple Choice Questions (MCQs)

Each question has four options. Choose the ONE best answer.

Section 1: Decision Making (4.1)

Q1. What is the purpose of control structures in a Python program?

- A) To store data in variables
 - B) To manage decision-making and repetition of tasks
 - C) To import libraries into a program
 - D) To define the colour of the output text
-

Q2. Which of the following is the correct syntax for a Python `if` statement?

- A) `if (condition) { code }`
 - B) `if condition: code`
 - C) `if condition then code`
 - D) `if: condition code`
-

Q3. In Python, what happens if the condition in an `if` statement is `False`?

- A) The program crashes with an error
- B) The program runs the indented block anyway

- C) Python skips the indented block and continues to the next line
 - D) The program stops completely
-

Q4. What will be the output of the following code?

```
temperature = 20
if temperature > 30:
    print("Hot day!")
```

- A) `Hot day!`
 - B) `False`
 - C) `0`
 - D) *(no output — the program prints nothing)*
-

Q5. What does `IndentationError` mean in Python?

- A) You used the wrong variable name
 - B) You forgot to add a colon after `if`
 - C) The indented block is missing or incorrectly spaced
 - D) The condition is written in the wrong order
-

Q6. Which symbol is used at the end of every `if`, `else`, `while`, and `for` line in Python?

- A) Semicolon `;`
 - B) Comma `,`
 - C) Full stop `.`
 - D) Colon `:`
-

Q7. What is the output of this code?

```
age = 18
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

- A) `Minor`
- B) `Adult`

- C) 18
 - D) No output
-

Q8. The `if-else` statement guarantees that:

- A) Both blocks always run
 - B) Neither block runs if the condition is False
 - C) Exactly one of the two blocks always runs
 - D) The `else` block runs only when the condition is True
-

Q9. In the following nested `if` code, what will be printed?

```
weather = "rainy"
temperature = 20
if weather == "rainy":
    if temperature < 15:
        print("Wear a raincoat!")
    else:
        print("Take an umbrella.")
else:
    print("Enjoy your day!")
```

- A) Wear a raincoat!
 - B) Take an umbrella.
 - C) Enjoy your day!
 - D) Nothing is printed
-

Q10. In a nested `if` statement, when does the inner `if` get checked?

- A) Always — before the outer `if`
 - B) Only when the outer `if` condition is `True`
 - C) Only when the outer `if` condition is `False`
 - D) At the same time as the outer `if`
-

Q11. What operator is used to check equality in a Python condition?

- A) `=`
- B) `==`

- C) `==`
 - D) `!=`
-

Q12. Which of the following is a valid Python condition?

- A) `if x = 5:`
 - B) `if x == 5:`
 - C) `if x === 5:`
 - D) `if x is equal to 5:`
-

Section 2: Looping Constructs (4.2)

Q13. A `while` loop checks its condition:

- A) Only once at the beginning
 - B) Only once at the end
 - C) Before every single iteration
 - D) After every two iterations
-

Q14. What will be the output of this code?

```
number = 1
while number < 4:
    print(number)
    number = number + 1
```

- A) `1 2 3 4`
 - B) `1 2 3`
 - C) `0 1 2 3`
 - D) `1 2 3 4 5`
-

Q15. What is an infinite loop?

- A) A loop that runs exactly 100 times
 - B) A loop whose condition never becomes `False`, so it runs forever
 - C) A loop with no body (no indented code inside)
 - D) A loop that counts backwards
-

Q16. Which keyboard shortcut is used to stop an infinite loop in Python?

- A) `Ctrl + Z`
 - B) `Ctrl + S`
 - C) `Ctrl + C`
 - D) `Ctrl + X`
-

Q17. What is the key difference between a `while` loop and a `for` loop?

- A) `while` loops can only count numbers; `for` loops can only read strings
 - B) `while` loops run based on a condition; `for` loops iterate over a sequence
 - C) `for` loops are faster than `while` loops
 - D) `while` loops do not need indentation
-

Q18. What will be the output of this code?

```
friends = ["Sami", "Raza", "Moosa"]
for friend in friends:
    print(friend)
```

- A) `Sami Raza Moosa` (on one line)
 - B) `["Sami", "Raza", "Moosa"]`
 - C) `Sami Raza Moosa` (each on a new line)
 - D) `0 1 2`
-

Q19. What does `range(5)` produce?

- A) Numbers 1, 2, 3, 4, 5
 - B) Numbers 0, 1, 2, 3, 4
 - C) Numbers 0, 1, 2, 3, 4, 5
 - D) Numbers 1, 2, 3, 4
-

Q20. What does `range(1, 6)` produce?

- A) 1, 2, 3, 4, 5, 6
- B) 0, 1, 2, 3, 4, 5
- C) 1, 2, 3, 4, 5

- D) 1, 2, 3, 4
-

Q21. What will `range(0, 10, 2)` produce?

- A) 0, 2, 4, 6, 8, 10
 - B) 0, 2, 4, 6, 8
 - C) 2, 4, 6, 8, 10
 - D) 1, 3, 5, 7, 9
-

Q22. What does the third argument in `range(start, stop, step)` control?

- A) The maximum value in the sequence
 - B) The number of repetitions
 - C) The jump size between each number
 - D) The data type of the numbers produced
-

Q23. What is the output of this code?

```
for i in range(2, 10, 3):  
    print(i, end=" ")
```

- A) 2 5 8 11
 - B) 2 4 6 8
 - C) 2 5 8
 - D) 3 6 9
-

Q24. What does `end=" "` do inside a `print()` statement?

- A) Adds an extra blank line after the output
 - B) Prints a space instead of moving to a new line
 - C) Ends the program after printing
 - D) Prints the word "end" before the value
-

Q25. In a nested loop, how many times does the inner loop run for each step of the outer loop?

- A) Once
- B) Twice

- C) It runs its full cycle from start to finish
 - D) The same number of times as the outer loop
-

Q26. What will be the output of this code?

```
for i in range(3):  
    for j in range(2):  
        print("*", end=" ")  
    print()
```

- A)
 ...
-
-

- B)

```
* *  
* *  
* *
```

- C)

- D)

```
* *
```

```
...
```

Q27. What is the correct three-ingredient formula for a safe `while` loop?

- A) A condition, a print statement, and an import
 - B) A starting value, a condition, and an update inside the loop
 - C) A variable, a function, and a return value
 - D) A list, an index, and a counter
-

Q28. Which loop is best suited when you know exactly how many times you want to repeat something?

- A) `while` loop

- B) `if-else` statement
 - C) `for` loop with `range()`
 - D) Nested `if`
-

Section 3: Libraries in Python (4.3)

Q29. What is a Python library?

- A) A place where Python books are stored
 - B) A collection of ready-made code that you can import and use
 - C) A special type of loop
 - D) A way to store multiple values in one variable
-

Q30. Which keyword is used to load a library in Python?

- A) `use`
 - B) `include`
 - C) `import`
 - D) `load`
-

Q31. What is the correct way to use the `sqrt` function from the `math` library?

- A) `sqrt(25)`
 - B) `math-sqrt(25)`
 - C) `math.sqrt(25)`
 - D) `import sqrt(25)`
-

Q32. What will `math.sqrt(49)` return?

- A) `49.0`
 - B) `7`
 - C) `7.0`
 - D) `2401`
-

Q33. Which library would you use to generate a random number between 1 and 100?

- A) `math`

- B) `statistics`
 - C) `random`
 - D) `os`
-

Q34. What does `random.randint(1, 10)` do?

- A) Always returns 5 (the midpoint)
 - B) Returns a random decimal number between 1 and 10
 - C) Returns a random whole number between 1 and 10, inclusive
 - D) Returns a list of numbers from 1 to 10
-

Q35. What does `statistics.mean([10, 20, 30])` return?

- A) `10`
 - B) `30`
 - C) `20.0`
 - D) `60`
-

Q36. What is the correct syntax to import only the `sqrt` function from the `math` library?

- A) `import math.sqrt`
 - B) `from math import sqrt`
 - C) `import sqrt from math`
 - D) `use math.sqrt`
-

Q37. What is the value of `math.pi`?

- A) `3.14`
 - B) `3.141592653589793`
 - C) `22/7`
 - D) `3`
-

Q38. Why do programmers use libraries instead of writing all code from scratch?

- A) Libraries make the code run slower but look better
- B) Libraries save time and effort by providing pre-written, tested tools

- C) Libraries are required by law in all Python programs
 - D) Libraries replace the need for variables and loops
-

Section 4: Lists in Python (4.4)

Q39. Which of the following correctly creates a list of three fruits in Python?

- A) `fruits = ("Mango", "Apple", "Banana")`
 - B) `fruits = ["Mango", "Apple", "Banana"]`
 - C) `fruits = {"Mango", "Apple", "Banana"}`
 - D) `fruits = <Mango, Apple, Banana>`
-

Q40. What is the index of the **first** item in a Python list?

- A) `1`
 - B) `-1`
 - C) `0`
 - D) It depends on the list
-

Q41. Given `fruits = ["Mango", "Apple", "Banana"]`, what does `fruits[1]` return?

- A) `"Mango"`
 - B) `"Apple"`
 - C) `"Banana"`
 - D) `1`
-

Q42. Given `fruits = ["Mango", "Apple", "Banana", "Guava"]`, what does `fruits[-1]` return?

- A) `"Mango"`
 - B) `"Apple"`
 - C) `"Banana"`
 - D) `"Guava"`
-

Q43. What error does Python raise if you try to access `fruits[10]` when the list only has 4 items?

- A) `SyntaxError`
 - B) `NameError`
 - C) `IndexError`
 - D) `ValueError`
-

Q44. How do you change the first item in `fruits = ["Mango", "Apple"]` to `"Orange"`?

- A) `fruits.change(0, "Orange")`
 - B) `fruits(0) = "Orange"`
 - C) `fruits[0] = "Orange"`
 - D) `fruits.replace("Mango", "Orange")`
-

Q45. Which method adds a new item to the **end** of a list?

- A) `.add()`
 - B) `.insert()`
 - C) `.push()`
 - D) `.append()`
-

Q46. What does `students.remove("Sara")` do if `"Sara"` appears twice in the list?

- A) Removes both occurrences of `"Sara"`
 - B) Removes the last occurrence of `"Sara"`
 - C) Removes the first occurrence of `"Sara"` only
 - D) Raises an error because there are two matching items
-

Q47. What does `numbers.sort()` do to a list of numbers?

- A) Reverses the list
 - B) Sorts in descending order (largest first)
 - C) Sorts in ascending order (smallest first)
 - D) Removes duplicates
-

Q48. What does `fruits.reverse()` do?

- A) Sorts the list alphabetically in reverse
 - B) Reverses the current order of items in the list
 - C) Removes the last item from the list
 - D) Converts all items to uppercase
-

Q49. What will `len(["Ahmed", "Sara", "Ali", "Hina"])` return?

- A) 3
 - B) 4
 - C) 5
 - D) 0
-

Q50. Which of the following creates an empty list?

- A) `my_list = {}`
 - B) `my_list = ()`
 - C) `my_list = []`
 - D) `my_list = ""`
-

Q51. What will be the output of this code?

```
numbers = [34, 12, 5, 89, 23]
numbers.sort()
print(numbers)
```

- A) `[89, 34, 23, 12, 5]`
 - B) `[5, 12, 23, 34, 89]`
 - C) `[34, 12, 5, 89, 23]`
 - D) `[12, 34, 5, 23, 89]`
-

Q52. Can a Python list contain items of different data types (e.g., numbers and strings together)?

- A) No — all items must be the same type
- B) Yes — a list can hold any mix of data types
- C) Only if the list has fewer than 5 items

- D) Only numbers are allowed in lists
-

Section 5: Testing and Debugging (4.5)

Q53. What is the purpose of **testing** in programming?

- A) To make the code run faster
 - B) To run the code with various inputs and verify it behaves correctly
 - C) To import the correct libraries
 - D) To format the output in a specific style
-

Q54. Which type of testing checks individual parts of the code (like one function) in isolation?

- A) Integration Testing
 - B) Functional Testing
 - C) Unit Testing
 - D) Regression Testing
-

Q55. Which type of testing ensures that fixing a new bug did not accidentally break existing features?

- A) Unit Testing
 - B) Functional Testing
 - C) Regression Testing
 - D) Integration Testing
-

Q56. What Python module is commonly used for unit testing?

- A) `random`
 - B) `unittest`
 - C) `statistics`
 - D) `math`
-

Q57. What is a **bug** in programming?

- A) A virus that attacks your computer
- B) An error or mistake in the code that causes it to behave incorrectly

- C) A type of loop that runs forever
 - D) A comment written in the code
-

Q58. Who found the first actual bug in a computer, and what was it?

- A) Alan Turing — a loose wire
 - B) Ada Lovelace — a calculation mistake
 - C) Grace Hopper — a real moth stuck inside a relay
 - D) Bill Gates — a corrupted file
-

Q59. What is the difference between a **syntax error** and a **logic error**?

- A) Syntax errors are harder to find; logic errors are easy
 - B) Syntax errors prevent the code from running; logic errors allow the code to run but produce wrong output
 - C) Logic errors crash the program; syntax errors produce wrong output
 - D) There is no difference — both cause the program to crash
-

Q60. Look at this code. What type of error does it contain?

```
if temperature > 30
    print("Hot!")
```

- A) Logic Error
 - B) Runtime Error
 - C) Syntax Error
 - D) IndexError
-

Q61. An infinite loop is typically caused by:

- A) A syntax error in the condition
 - B) The loop variable never being updated, so the condition never becomes False
 - C) Using `for` instead of `while`
 - D) Forgetting to import a library
-

Q62. Which debugging technique involves adding `print()` statements at different points in the code to check variable values?

- A) Unit Testing
 - B) Trace Through Manually
 - C) Print Statement Debugging
 - D) Regression Testing
-

Q63. When Python shows a `SyntaxError`, what useful information does the error message always include?

- A) The name of the programmer who made the error
 - B) The line number where the error was found
 - C) A suggestion for the correct code
 - D) The value of all variables at the time of error
-

Q64. What is the best first step when you see an error message in Python?

- A) Delete all the code and start over
 - B) Ignore it — error messages are not useful
 - C) Read the error message carefully — it tells you the line number and type of error
 - D) Restart your computer
-

Q65. What will happen when this code runs?

```
number = 10
while number > 0:
    print(number)
    number = number + 1
```

- A) It prints 10, 9, 8 ... down to 1 and stops
 - B) It prints nothing
 - C) It prints 10, 11, 12 ... and runs forever (infinite loop)
 - D) It prints only `10` once
-
-

MCQ Answer Key

Q	Answer	Q	Answer	Q	Answer	Q	Answer	Q	Answer
1	B	14	B	27	B	40	C	53	B
2	B	15	B	28	C	41	B	54	C
3	C	16	C	29	B	42	D	55	C
4	D	17	B	30	C	43	C	56	B
5	C	18	C	31	C	44	C	57	B
6	D	19	B	32	C	45	D	58	C
7	B	20	C	33	C	46	C	59	B
8	C	21	B	34	C	47	C	60	C
9	B	22	C	35	C	48	B	61	B
10	B	23	C	36	B	49	B	62	C
11	C	24	B	37	B	50	C	63	B
12	B	25	C	38	B	51	B	64	C
13	C	26	B	39	B	52	B	65	C

PART B — Short Questions

Write a clear, concise answer for each question. Aim for 2–5 sentences. Include code where the question asks for it.

Section 1: Decision Making

Q1. What are control structures? Name the two main types discussed in this chapter.

Answer: Control structures are programming tools that control the flow of a program — they determine which lines of code run and when. Without them, a program simply runs from top to bottom once. The two main types are **decision making** (using `if`, `if-else`, and nested `if`) and **looping** (using `while` and `for`).

Q2. What is the `if` statement? Write its syntax in Python.

Answer: The `if` statement is the most basic decision in Python. It checks a condition — a question that is either `True` or `False` — and runs the indented block of code **only if** the condition is `True`. If the condition is `False`, Python skips the block entirely.

```
if condition:
    code to run if condition is True
```

Q3. What is indentation in Python? Why is it important?

Answer: Indentation means pushing code inward (using 4 spaces or one Tab). In Python, indentation defines which lines of code **belong to** a block — for example, which lines are inside an `if` statement or a loop. Indentation is **not optional** in Python. If you skip it, Python raises an `IndentationError` and the program will not run.

Q4. What is the difference between `=` and `==` in Python?

Answer: A single `=` is the **assignment operator** — it gives a value to a variable (e.g., `x = 5` stores the number 5 in `x`). A double `==` is the **equality operator** — it checks whether two values are equal and returns `True` or `False` (e.g., `x == 5` asks "is x equal to 5?"). Using `=` inside an `if` condition is a very common mistake — always use `==` to compare.

Q5. What is the `if-else` statement? How is it different from a plain `if` statement?

Answer: The `if-else` statement handles **both sides** of a decision. If the condition is `True`, the first block runs. If the condition is `False`, the `else` block runs instead. Unlike a plain `if` statement (which does nothing when the condition is `False`), `if-else` guarantees that **exactly one block** always runs — the program never skips both.

Q6. What is a nested condition? Give one real-life example.

Answer: A nested condition is an `if` statement placed **inside** another `if` statement. The inner condition is only checked if the outer condition is `True`. A real-life example: a school gate guard first checks if you have an ID card. **Only if** you have an ID does the guard then check if you are wearing the proper uniform. One check sits inside another.

Q7. Write a Python program using a nested `if` that checks if a student has a lab pass AND if it is not a holiday, and prints an appropriate message for each case.

Answer:

```
has_lab_pass = True
is_holiday = False

if has_lab_pass:
    if not is_holiday:
        print("Access granted. Welcome to the lab!")
    else:
        print("Lab is closed today – it's a holiday.")
else:
    print("Access denied. You need a lab pass.")
```

Section 2: Looping Constructs

Q8. What is a `while` loop? Write its syntax.

Answer: A `while` loop repeats a block of code **as long as** a condition is `True`. Before every repetition, Python checks the condition. The moment the condition becomes `False`, the loop stops. A `while` loop needs a starting value, a condition, and an update inside the body — otherwise, the condition may never become `False`.

```
while condition:
    code to run while condition is True
```

Q9. What are the three essential ingredients of a safe `while` loop? What happens if one is missing?

Answer: A safe `while` loop needs:

1. A **starting value** — to give the condition something to check.
2. A **condition** — the question Python checks before each round.
3. An **update** — a line inside the loop that changes the value so the condition eventually becomes `False`.

If the update is missing, the condition can **never** become `False`, and the loop runs forever — this is called an **infinite loop**.

Q10. What is an infinite loop? How do you stop one?

Answer: An infinite loop is a `while` loop whose condition never becomes `False`, so it keeps running forever without stopping. This usually happens when the programmer forgets to update the variable inside the loop. To stop an infinite loop, press **Ctrl + C** on the keyboard. Then go back and fix the loop condition or update line.

Q11. What is a `for` loop? How is it different from a `while` loop?

Answer: A `for` loop repeats code **for each item in a sequence** — like a list or a `range()` of numbers. It automatically moves from one item to the next and stops when the sequence is finished. A `while` loop, by contrast, repeats based on a **condition** that is checked before every round. A `for` loop is better when you know exactly how many times you want to repeat; a `while` loop is better when you repeat until a condition changes.

Q12. What does the `range()` function do? Give an example with all three arguments (`start`, `stop`, `step`).

Answer: The `range()` function generates a sequence of numbers that can be used in a `for` loop. It takes up to three arguments: where to **start**, where to **stop** (not including the stop number), and the **step** (how much to jump each time).

```
for i in range(1, 10, 2):
    print(i, end=" ")
# Output: 1 3 5 7 9
```

Q13. What is `range(5)`? What numbers does it produce? Many beginners expect it to start from 1 — why is that wrong?

Answer: `range(5)` produces the numbers **0, 1, 2, 3, 4** — five numbers starting from zero. Beginners often expect it to start from 1 because we count from 1 in daily life. But Python always starts `range()` from **0** when no start value is given, because Python counts positions starting from 0 (just like list indexes).

Q14. What is a nested loop? Write a Python program that uses nested loops to print a 3×4 pattern of asterisks.

Answer: A nested loop is a loop placed **inside** another loop. For every single iteration of the outer loop, the inner loop runs completely from start to finish.

```
for i in range(3):           # Outer loop runs 3 times (rows)
    for j in range(4):       # Inner loop runs 4 times (columns)
        print("*", end=" ")
    print()                  # Move to next line after each row
```

Output:

```
* * * *
* * * *
* * * *
```

Q15. What does `end=" "` do in a `print()` statement? Give an example.

Answer: By default, every `print()` in Python ends with a **newline** — moving the cursor to the next line. Adding `end=" "` replaces the newline with a **space**, so the next print continues on the same line.

```
for i in range(1, 6):
    print(i, end=" ")
# Output: 1 2 3 4 5
```

Section 3: Libraries

Q16. What is a Python library? Why do programmers use them?

Answer: A Python library (also called a module) is a collection of **ready-made, pre-written code** that anyone can import and use. Programmers use libraries because they save time — instead of writing a complex function like square root calculation from scratch, you can import `math` and use `math.sqrt()` immediately. Libraries are written and tested by experts, so they are reliable.

Q17. Write Python code to import the `math` library and use it to calculate the area of a circle with radius 5.

Answer:

```
import math

radius = 5
area = math.pi * math.pow(radius, 2)
print("Area of circle:", area)
# Output: Area of circle: 78.53981633974483
```

Q18. What is the difference between `import math` and `from math import sqrt`?

Answer: `import math` loads the **entire** `math` library. You then call functions using the format `math.sqrt()`. `from math import sqrt` imports **only** the `sqrt` function from the library. After this, you can call it simply as `sqrt()` without the `math.` prefix. The second style is shorter and useful when you only need one or two specific tools from a large library.

Q19. What does `random.randint(1, 10)` do? Will it always return the same number?

Answer: `random.randint(1, 10)` generates a **random whole number** between 1 and 10, including both 1 and 10. No, it will **not** always return the same number — that is the point of using the `random` library. Every time you run the code, it may return a different number. This is useful for games, simulations, and testing.

Section 4: Lists

Q20. What is a list in Python? How do you create one?

Answer: A list is a variable that holds **multiple values** in a specific order, all in one place. Lists are created using **square brackets** with items separated by commas. Lists can hold any type of data — numbers, strings, or a mix.

```
fruits = ["Mango", "Apple", "Banana"]
numbers = [10, 25, 3, 88]
```

Q21. Why does Python list indexing start from 0 instead of 1?

Answer: Python counts positions from **zero** because of how computers store data in memory — the first slot in memory has an offset of 0 from the starting address. This is a convention used in most programming languages. So the first item is at index `0`, the second at `1`, and so on. It surprises beginners, but it quickly becomes natural with practice.

Q22. What is a negative index in a Python list? Give an example.

Answer: A negative index counts from the **end** of the list. `list[-1]` gives the **last** item, `list[-2]` gives the second to last, and so on. This is useful when you need the last few items without knowing the exact length.

```
fruits = ["Mango", "Apple", "Banana", "Guava"]
print(fruits[-1])    # Output: Guava
print(fruits[-2])    # Output: Banana
```

Q23. What is `IndexError`? When does it occur?

Answer: An `IndexError` occurs when you try to access a list item at an index that **does not exist**. For example, if a list has 4 items (indexes 0 to 3) and you try `list[10]`, Python raises `IndexError: list index out of range`. This is Python's way of saying: "There is no item at that position."

Q24. What do the following list methods do? Write one example for each:

`.append()`, `.remove()`, `.sort()`, `.reverse()`.

Answer:

- `.append(item)` — Adds an item to the **end** of the list.
python

```
students = ["Ali", "Sara"]
students.append("Ahmed")
# Result: ["Ali", "Sara", "Ahmed"]
```
 - `.remove(item)` — Removes the **first** occurrence of the item.
python

```
students.remove("Sara")
# Result: ["Ali", "Ahmed"]
```
 - `.sort()` — Sorts the list in **ascending** order.
python

```
numbers = [5, 2, 9, 1]
numbers.sort()
# Result: [1, 2, 5, 9]
```
 - `.reverse()` — Reverses the current order of items.
python

```
numbers.reverse()
# Result: [9, 5, 2, 1]
```
-

Q25. Write Python code that creates a list of five numbers, adds a new number, removes the smallest one, then prints the final sorted list.

Answer:

```
numbers = [15, 42, 7, 30, 88]
numbers.append(55)           # Add a new number
numbers.remove(7)            # Remove the smallest number
numbers.sort()               # Sort in ascending order
print(numbers)
# Output: [15, 30, 42, 55, 88]
```

Section 5: Testing and Debugging

Q26. What is testing in programming? Why is it important?

Answer: Testing is the process of running a program with **various inputs** to check that it produces the correct output in all situations. It is important because a program can run without errors and still give wrong answers. Testing catches these problems before the program is used in the real world, where wrong outputs can cause serious problems.

Q27. Name the four types of testing mentioned in this chapter and describe each in one sentence.

Answer:

1. **Unit Testing** — Tests individual functions or small parts of the code in isolation.
2. **Integration Testing** — Checks that different parts of the code work correctly **together**.
3. **Functional Testing** — Validates that the software does what the **user expects** it to do.
4. **Regression Testing** — Ensures that fixing a new bug did **not break** any existing features.

Q28. What is a test case? Give an example of three test cases for a function that multiplies two numbers.

Answer: A test case is a specific **input** paired with the **expected output** that you use to verify your code. For a `multiply(a, b)` function:

Test Case	Input	Expected Output	Purpose
1	<code>multiply(3, 4)</code>	<code>12</code>	Normal case
2	<code>multiply(0, 5)</code>	<code>0</code>	Edge case with zero
3	<code>multiply(-2, 6)</code>	<code>-12</code>	Negative number case

Q29. What is debugging? Who coined the word "bug" and how?

Answer: Debugging is the process of **finding and fixing errors** in code. The word "bug" comes from **Grace Hopper**, an engineer at Harvard University. In 1947, while working on the Harvard Mark II computer, her team found a **real moth** stuck inside an electrical relay that was causing errors. She taped it into the logbook with the note *"First actual case of bug being found."* Since then, programming errors have been called "bugs."

Q30. What is the difference between a syntax error and a logic error? Give one example of each.

Answer:

- **Syntax Error** — The code is grammatically wrong. Python cannot understand it and **refuses to run** at all. Example: forgetting the colon after `if`.

```
python
if x > 5      # SyntaxError: expected ':'
    print("Yes")
```

- **Logic Error** — The code is grammatically correct (Python runs it without crashing), but the **output is wrong** because the programmer's logic was incorrect. Example: using `+` instead of `-` in a countdown loop, causing it to count up instead.

```
python
number = 10
while number > 0:
    number = number + 1  # Logic Error: should be -1
```

Q31. List three debugging techniques and explain each one briefly.

Answer:

1. **Print Statement Debugging** — Add `print()` lines inside your code to display the value of variables at different stages. This lets you see exactly where a variable gets the wrong value.
 2. **Read the Error Message** — When Python gives an error, read it carefully. It tells you the **line number** and the **type of error**. Go to that line first.
 3. **Trace Through Manually** — Go through your code line by line on paper, pretending to be Python. Write down the value of each variable at each step to find where the logic goes wrong.
-
-

PART C — Long Questions

Write detailed, structured answers. Include code examples, trace tables, and explanations where asked. Each question carries significant marks.

L.Q. 1 — Decision Making Statements

- (a) Explain what decision making means in Python programming. Why is it important?
- (b) Write the syntax and explain the working of: (i) the `if` statement, (ii) the `if-else` statement, and (iii) a nested `if` statement.
- (c) Write a Python program that asks the user to enter their marks (out of 100) and prints the following result based on the marks:
- 80 and above → `"Grade A"`
 - 60 to 79 → `"Grade B"`
 - 40 to 59 → `"Grade C"`
 - Below 40 → `"Fail"`

Answer:

(a) Decision making in Python means writing code that can **choose between different actions** based on a condition. Without decision making, a program runs the same lines every time — no matter what values the user enters or what situation arises. Decision making makes programs **intelligent and responsive**. For example, a banking app checks your account balance before allowing a withdrawal; a game checks whether your health is zero before ending the match.

(b)

(i) **`if` Statement:**

Checks one condition. If it is `True`, the indented block runs. If `False`, Python skips the block.

```
# Syntax
if condition:
    code to run if True

# Example
temperature = 35
if temperature > 30:
    print("It's a hot day!")
# Output: It's a hot day!
```

(ii) **if-else** Statement:

Handles both outcomes. If the condition is **True**, the first block runs. If **False**, the **else** block runs instead. Exactly one block always runs.

```
# Syntax
if condition:
    code if True
else:
    code if False

# Example
temperature = 15
if temperature > 30:
    print("Hot day!")
else:
    print("Not a hot day.")
# Output: Not a hot day.
```

(iii) Nested **if** Statement:

An **if** inside another **if**. The inner condition is only checked if the outer condition is already **True**.

```
# Syntax
if condition1:
    if condition2:
        code if both are True
    else:
        code if only condition1 is True
else:
    code if condition1 is False

# Example
weather = "rainy"
temperature = 10
if weather == "rainy":
    if temperature < 15:
        print("Wear a raincoat!")
    else:
        print("Take an umbrella.")
else:
    print("Enjoy your day!")
# Output: Wear a raincoat!
```

(c)

```
marks = int(input("Enter your marks (0-100): "))

if marks >= 80:
    print("Grade A")
else:
    if marks >= 60:
        print("Grade B")
    else:
        if marks >= 40:
            print("Grade C")
        else:
            print("Fail")
```

Sample Output (if marks = 73):

```
Grade B
```

L.Q. 2 — Looping Constructs

- (a) Explain the `while` loop and the `for` loop. What is the key difference between them?
- (b) What is the `range()` function? Explain its three forms with examples.
- (c) Write a Python program using a `while` loop that simulates a water tank filling at 10 litres per minute until it reaches 100 litres. Print the water level after each minute.
- (d) Write a Python program using a `for` loop and `range()` to print the multiplication table of 7 (from 7×1 to 7×10).

Answer:

(a)

while loop: Repeats a block of code **as long as a condition is True**. It checks the condition before every iteration.

```
# Syntax
while condition:
    code to repeat
```

for loop: Repeats code **for each item in a sequence**. It automatically moves through every item and stops when the sequence ends.

```
# Syntax
for variable in sequence:
    code to repeat
```

Key difference: A `while` loop is condition-driven (you control when it stops). A `for` loop is sequence-driven (the sequence controls how many times it runs).

(b)

`range()` generates a sequence of numbers for use in `for` loops.

Form	Meaning	Example	Output
<code>range(stop)</code>	0 up to <code>stop - 1</code>	<code>range(4)</code>	0, 1, 2, 3
<code>range(start, stop)</code>	<code>start</code> up to <code>stop - 1</code>	<code>range(2, 6)</code>	2, 3, 4, 5
<code>range(start, stop, step)</code>	<code>start</code> to <code>stop - 1</code> , jumping by <code>step</code>	<code>range(1, 10, 3)</code>	1, 4, 7

Important: `range()` always stops **before** the stop number.

(c)

```
water_level = 0

while water_level < 100:
    water_level = water_level + 10
    print("Water level:", water_level, "litres")

print("Tank is full!")
```

Output:

```
Water level: 10 litres
Water level: 20 litres
Water level: 30 litres
Water level: 40 litres
Water level: 50 litres
Water level: 60 litres
Water level: 70 litres
Water level: 80 litres
Water level: 90 litres
Water level: 100 litres
Tank is full!
```

Trace Table:

Iteration	water_level before	Condition < 100 ?	Output	water_level after
1	0	True	10 litres	10
2	10	True	20 litres	20
...	...	True	...	...
10	90	True	100 litres	100
11	100	False	Loop ends	—

(d)

```
for i in range(1, 11):
    print(f"7 × {i} = {7 * i}")
```

Output:

```
7 × 1 = 7
7 × 2 = 14
7 × 3 = 21
7 × 4 = 28
7 × 5 = 35
7 × 6 = 42
7 × 7 = 49
7 × 8 = 56
7 × 9 = 63
7 × 10 = 70
```

L.Q. 3 — Nested Loops and Patterns

(a) What is a nested loop? Explain with an analogy.

(b) Write a Python program using nested loops that prints the following right-angled triangle pattern of 5 rows:

```
*
* *
* * *
* * * *
* * * * *
```

(c) Trace through your nested loop program manually — show the value of the outer loop variable and inner loop variable at every step for the first 3 rows.

Answer:

(a) A nested loop is a loop placed **inside another loop**. For every **single step** of the outer loop, the inner loop runs its **complete cycle** from beginning to end.

Analogy: Think of a school timetable. The outer loop is the days of the week (Monday to Friday). For each day, the inner loop goes through all 6 periods. When Monday's 6 periods are finished, the outer loop moves to Tuesday — and the inner loop runs all 6 periods again.

(b)

```
n = 5

for i in range(1, n + 1):      # Outer loop: i = 1, 2, 3, 4, 5
    for j in range(i):        # Inner loop: runs 'i' times
        print("*", end=" ")
    print()                   # New line after each row
```

Output:

```
*
* *
* * *
* * * *
* * * * *
```

(c) Manual Trace — First 3 Rows:

Outer loop i	Inner loop j values	Stars printed	Line ends
1	j = 0	*	→ new line
2	j = 0, 1	* *	→ new line
3	j = 0, 1, 2	* * *	→ new line

Key observation: When **i = 3**, the inner loop runs for **j = 0**, then **j = 1**, then **j = 2** — three times total. Each time it prints one *****. Then **print()** moves to the next line. The outer loop then moves to **i = 4**.

L.Q. 4 — Python Libraries

(a) What is a Python library? Why are libraries important in programming? Use a real-world analogy to explain.

(b) Write Python code to demonstrate the use of the `math`, `random`, and `statistics` libraries. For each, show one practical example with the expected output.

(c) What is the difference between `import math` and `from math import sqrt, pi`? When would you prefer one over the other?

Answer:

(a) A Python library (or module) is a **collection of ready-made, pre-written code** that any programmer can import and use. Libraries are important because they save enormous time — a programmer does not need to build every tool from scratch.

Analogy: Think of a library like a **toolbox**. A plumber does not forge a new wrench from raw metal every time they need to tighten a pipe. They reach into their toolbox, pick up the wrench, and get to work. Similarly, a Python programmer does not write square root functions from scratch — they `import math` and use `math.sqrt()` immediately.

(b)

`math` library — for mathematical calculations:

```
import math

print("Square root of 64:", math.sqrt(64))
print("Value of pi:", math.pi)
print("2 to the power of 10:", math.pow(2, 10))
```

```
Square root of 64: 8.0
Value of pi: 3.141592653589793
2 to the power of 10: 1024.0
```

`random` library — for random number generation:

```
import random

number = random.randint(1, 100)
print("Random number:", number)
```

```
Random number: 47 (this will vary each time)
```

`statistics` library — for data analysis:

```
import statistics

scores = [55, 72, 88, 91, 63, 79]
print("Mean score:", statistics.mean(scores))
```

(c)

Method	What it imports	How you call functions	Best used when
<code>import math</code>	The entire math library	<code>math.sqrt()</code> , <code>math.pi</code>	You need many functions from the library
<code>from math import sqrt, pi</code>	Only <code>sqrt</code> and <code>pi</code>	<code>sqrt()</code> , <code>pi</code> (no prefix needed)	You only need one or two specific tools

Preference:

- Use `import math` for clarity and to avoid naming conflicts with your own variables.
- Use `from math import sqrt` for shorter code when you know exactly which tools you need and your variable names do not clash with them.

L.Q. 5 — Lists in Python

- (a) What is a list in Python? What makes lists different from regular variables?
- (b) Explain list indexing in Python. Why does it start at 0? What are negative indexes?
- (c) Write a complete Python program that:
1. Creates a list of 5 student names
 2. Prints the full list
 3. Adds a new student at the end
 4. Removes one student by name
 5. Sorts the list alphabetically
 6. Prints the final list and the total number of students

Answer:

(a) A Python **list** is a variable that holds **multiple values** in a specific order, all under one name. A regular variable holds only **one value** at a time (e.g., `name = "Ali"`). A list holds **many values** in numbered positions (e.g., `names = ["Ali", "Sara", "Ahmed"]`). Lists are enclosed in square brackets and items are separated by commas.

(b) Every item in a list has an **index** — a number that identifies its position.

- Indexing starts at **0** (not 1) because computers count memory positions from zero. The first slot has an offset of 0 from the start of the list in memory.
- **Positive indexes** count from the start: `list[0]` = first item, `list[1]` = second, etc.
- **Negative indexes** count from the **end**: `list[-1]` = last item, `list[-2]` = second to last.

Index diagram:

```
fruits = ["Mango", "Apple", "Banana", "Guava"]
Positive:  [0]      [1]      [2]      [3]
Negative:  [-4]     [-3]     [-2]     [-1]
```

(c)

```
# Step 1: Create the list
students= ["Ahmed", "Sara", "Ali", "Hina", "Usman"]

# Step 2: Print the full list
print("Original list:", students)

# Step 3: Add a new student
students.append("Fatima")
print("After adding Fatima:", students)

# Step 4: Remove one student
students.remove("Sara")
print("After removing Sara:", students)

# Step 5: Sort alphabetically
students.sort()
print("Sorted list:", students)

# Step 6: Print final list and count
print("Final class list:", students)
print("Total number of students:", len(students))
```

Output:

```
Original list: ['Ahmed', 'Sara', 'Ali', 'Hina', 'Usman']
After adding Fatima: ['Ahmed', 'Sara', 'Ali', 'Hina', 'Usman', 'Fatima']
After removing Sara: ['Ahmed', 'Ali', 'Hina', 'Usman', 'Fatima']
Sorted list: ['Ahmed', 'Ali', 'Fatima', 'Hina', 'Usman']
Final class list: ['Ahmed', 'Ali', 'Fatima', 'Hina', 'Usman']
Total number of students: 5
```

L.Q. 6 — Testing and Debugging

(a) Explain the concept of **testing** in programming. Name and describe all four types of testing from this chapter.

(b) Explain the concept of **debugging**. What is the historical origin of the word "bug"?

(c) Identify the **type of error**, explain the **cause**, and write the **corrected code** for each of the following two programs:

Program 1 (Broken):

```
score = 85
if score >= 50
    print("Pass")
else:
    print("Fail")
```

Program 2 (Broken):

```
total = 0
for i in range(1, 6):
    total = total - i
print("Sum:", total)
```

(d) Write Python code to demonstrate print-statement debugging on a simple program that calculates the average of three numbers.

Answer:

(a)

Testing is the process of running a program with various inputs — including unusual and extreme ones — to verify that the program produces the correct output in every situation. A program that runs without crashing is **not** automatically correct. Testing is what checks whether the logic is right.

Four types:

1. **Unit Testing** — Tests a single function or small piece of code in isolation. Uses Python's `unittest` module. Purpose: verify that one specific tool works correctly before combining it with others.
2. **Integration Testing** — Tests how **multiple parts** of the program work together. Example: after testing the login function and the database function separately, test whether they communicate correctly.
3. **Functional Testing** — Tests from the **user's perspective**. Does the program do what the user expects? Example: does clicking "Submit" actually save the form to the database?

4. **Regression Testing** — After making a change or fix to the code, runs all previous tests again to make sure the change did not **accidentally break** something that was working before.

(b)

Debugging is the process of **finding and fixing errors (bugs)** in a program. It involves reading error messages, tracing through code, adding print statements, and identifying whether errors are in grammar (syntax) or logic.

Historical origin: On September 9th, 1947, engineers working on the **Harvard Mark II** computer — led by **Grace Hopper** — found that the machine was giving wrong results. After extensive searching, they discovered a **real moth** stuck inside one of the electrical relays. Grace Hopper removed the moth and taped it into the team's logbook with the note: *"First actual case of bug being found."* From that day, programming errors have been called **"bugs"** and fixing them is called **"debugging."**

(c)

Program 1 Analysis:

```
# BROKEN code:
score = 85
if score >= 50
    print("Pass")
else:
    print("Fail")
```

Error Type: `SyntaxError`

Cause: The `if` line is missing a **colon** `:` at the end. Every `if`, `else`, `while`, and `for` line in Python must end with a colon.

Error message Python gives:

```
SyntaxError: expected ':'
```

Corrected Code:

```
score = 85
if score >= 50:      # ← Colon added here
    print("Pass")
else:
    print("Fail")
```

Output:

Pass

Program 2 Analysis:

```
# BROKEN code:
total = 0
for i in range(1, 6):
    total = total - i
print("Sum:", total)
```

Error Type: Logic Error

Cause: The code uses subtraction (`total - i`) instead of addition (`total + i`). Python runs the code without any errors, but the result is wrong. Instead of summing $1+2+3+4+5 = 15$, it calculates $0-1-2-3-4-5 = -15$.

There is no error message — this is what makes logic errors dangerous.

Corrected Code:

```
total = 0
for i in range(1, 6):
    total = total + i    # ← Changed - to +
print("Sum:", total)
```

Output:

```
Sum: 15
```

(d) Print-Statement Debugging Example:

```
# Program to calculate average of three numbers
num1 = 10
num2 = 20
num3 = 30

# Debugging: print each variable to confirm values are correct
print("DEBUG - num1:", num1)
print("DEBUG - num2:", num2)
print("DEBUG - num3:", num3)

total = num1 + num2 + num3

# Debugging: print total before division
print("DEBUG - total:", total)

average = total / 3

# Debugging: print average before final output
print("DEBUG - average:", average)

print("Average:", average)
```

Output:

```
DEBUG - num1: 10
DEBUG - num2: 20
DEBUG - num3: 30
DEBUG - total: 60
DEBUG - average: 20.0
Average: 20.0
```

Purpose: Each `DEBUG` print line lets you see exactly what value a variable holds at that moment. If any `DEBUG` line shows a wrong value, you have found where the bug is — without guessing.

L.Q. 7 — Combined Program (Comprehensive)

Write a complete Python program that does all of the following:

1. Creates a list of 5 numbers entered manually in the code.
2. Uses a `for` loop to print each number and its square.
3. Uses a `while` loop to find and print the sum of all numbers in the list.
4. Imports the `math` library and prints the square root of the largest number.
5. Sorts the list and prints it.

Answer:

```

import math

# Step 1: Create a list of 5 numbers
numbers = [16, 9, 25, 4, 36]

# Step 2: for loop – print each number and its square
print("--- Number and its Square ---")
for num in numbers:
    print(f"{num} squared = {num * num}")

# Step 3: while loop – find the sum
print("\n--- Sum using while loop ---")
total = 0
index = 0

while index < len(numbers):
    total = total + numbers[index]
    index = index + 1

print("Sum of all numbers:", total)

# Step 4: Square root of the largest number using math library
largest = numbers[0]
for num in numbers:
    if num > largest:
        largest = num

print("\n--- Square Root of Largest Number ---")
print("Largest number:", largest)
print("Square root:", math.sqrt(largest))

# Step 5: Sort and print the list
numbers.sort()
print("\n--- Sorted List ---")
print(numbers)

```

Output:


```
--- Number and its Square ---  
16 squared = 256  
9 squared = 81  
25 squared = 625  
4 squared = 16  
36 squared = 1296  
  
--- Sum using while loop ---  
Sum of all numbers: 90  
  
--- Square Root of Largest Number ---  
Largest number: 36  
Square root: 6.0  
  
--- Sorted List ---  
[4, 9, 16, 25, 36]
```

Quick Reference Summary

Topic	Key Term	One-line Definition
4.1	Control Structure	Code that controls the flow — decisions and loops
4.1.1	<code>if</code> statement	Runs a block only when a condition is True
4.1.2	<code>if-else</code> statement	Always runs one of two blocks based on a condition
4.1.3	Nested condition	An <code>if</code> inside another <code>if</code>
4.2.1	<code>while</code> loop	Repeats while a condition is True
4.2.1	Infinite loop	A loop whose condition never becomes False
4.2.2	<code>for</code> loop	Repeats for each item in a sequence
4.2.3	<code>range()</code>	Generates a sequence of numbers
4.2.4	Nested loop	A loop inside another loop
4.3	Library / Module	A collection of ready-made, importable code
4.4	List	An ordered, changeable collection of values
4.4.2	Index	A position number used to access a list item
4.4.2	IndexError	Error when accessing a list position that does not exist
4.5.1	Testing	Running code with various inputs to verify correctness
4.5.2	Debugging	Finding and fixing errors in code
4.5.2	Syntax Error	Grammar mistake — Python cannot run the code
4.5.2	Logic Error	Thinking mistake — code runs but gives wrong output
4.5.2	Bug	An error in code (named after a real moth found in 1947)

End of Chapter 4 Question Bank

Total Questions: 65 MCQs | 31 Short Questions | 7 Long Questions